# Veri Seti Tanımı

## Banknot Sahteciliği Tespiti Veri Seti

Projemde veri kaynağı olarak UCI Machine Learning Repository üzerinden erişilebilen Banknote Authentication (Banknot Sahteciliği Tespiti) setini kullandım.
Bağlantı: https://archive.ics.uci.edu/dataset/267/banknote+authentication

Veri setinin yapısı aslında ilk bakışta akla gelenden biraz farklı; içinde doğrudan banknot fotoğraf arı yer almıyor. İşlemler, paraların yüksek çözünürlüklü görsellerinden Wavelet Transform (Dalgacık Dönüşümü) tekniğiyle çıkarılmış özellikler üzerinden yürüyor.

Bu işlem sonucunda her para için varyans, çarpıklık, basıklık ve entropi olmak üzere 4 temel sayısal değer elde edilmiş. Yani paranın üzerindeki o ince desenler, asimetri ve dokusal karmaşıklık tamamen matematiğe dökülmüş diyebiliriz. Setin boyutu çok devasa değil, toplamda 1372 adet verimiz var. Hedef değişken olarak da elimizdeki örneğin orijinal mi (1) yoksa sahte mi (0) olduğunu gösteren klasik bir etiketleme sistemi kullanılmış. Bu yüzden İkili Sınıf andırma Modeli kullandım.

# Model Mimarisi ve Kod

```
# Gerekli kütüphanelerin indirilmesi
# !pip install jupyter ipykernel pandas numpy matplotlib seaborn
scikit-learn tensorflow

# Gerekli kütüphanelerin kullanılması
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score, roc_auc_score, roc_curve
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam, S D
from tensorflow.keras.callbacks import EarlyStopping

# NumPy ve  ensorFlow'daki rastgele değerlerin her çalıştırmada aynı
çıkması için sabitledim.
np.random.seed(42)
tf.random.set_seed(42)
```

```python
# Veri setinde kullanılacak sütun isimleri
column_names = ["Variance", "Skewness", "Kurtosis", "Entropy",
"Class"]

# Veri setini yani banknotu dosyadan okuyorum ve sütun isimlerini
atıyorum.
df = pd.read_csv("data_banknote_authentication.txt",
names=column_names)

print(df.shape) #df satır ve sütun sayısı

print(df.head()) #ilk 5 satır

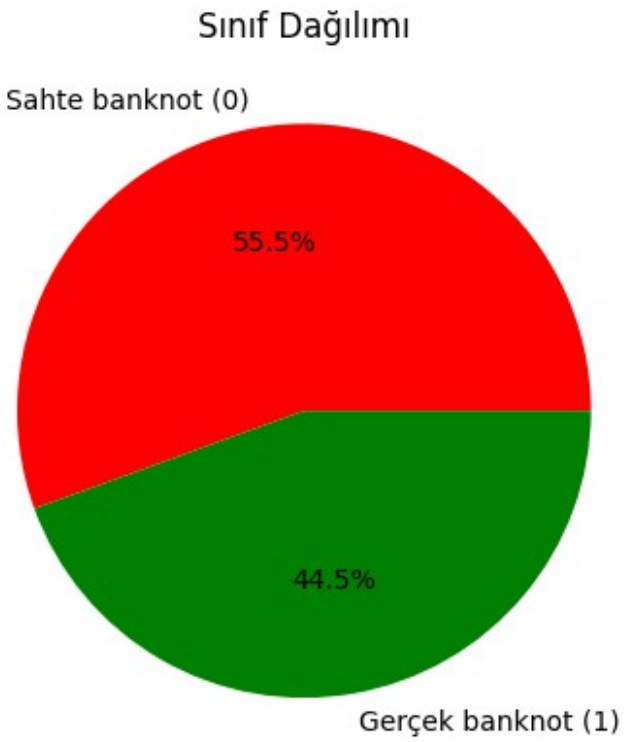
print(df['Class'].value_counts()) # Sınıf dağılımına bakarak gerçek ve
sahte banknot sayısını görüyorum
```

```
(1372, 5)
   Variance  Skewness  Kurtosis  Entropy  Class
0   3.62160    8.6661   -2.8073 -0.44699      0
1   4.54590    8.1674   -2.4586 -1.46210      0
2   3.86600   -2.6383    1.9242  0.10645      0
3   3.45660    9.5228   -4.0112 -3.59440      0
4   0.32924   -4.4552    4.5718 -0.98880      0
Class
0    762
1    610
Name: count, dtype: int64
```

```python
# 762 tane sahte, 610 tane gerçek banknot var.
# 0 sahte | 1 gerçek


# Sınıf dağılımını net görmek için pasta grafiği oluşturuyorum.
df['Class'].value_counts().plot( #sınıfları sayıyor her değerden kaç
tane var
    kind='pie', #pasta çiz
    autopct='%1.1f%%', # yüzdelik değerleri göster
    labels=['Sahte banknot (0)', 'Gerçek banknot (1)'],
    colors=['red', 'green']
)
plt.title('Sınıf Dağılımı') #grafik başlığı
plt.ylabel('') # “y eksenindeki başlığı kaldırdım

Text(0, 0.5, '')
```




```
# Değerlerinin dağılımını görmek için histogram grafiği oluşturuyorum.
plt.figure(figsize=(12, 8))

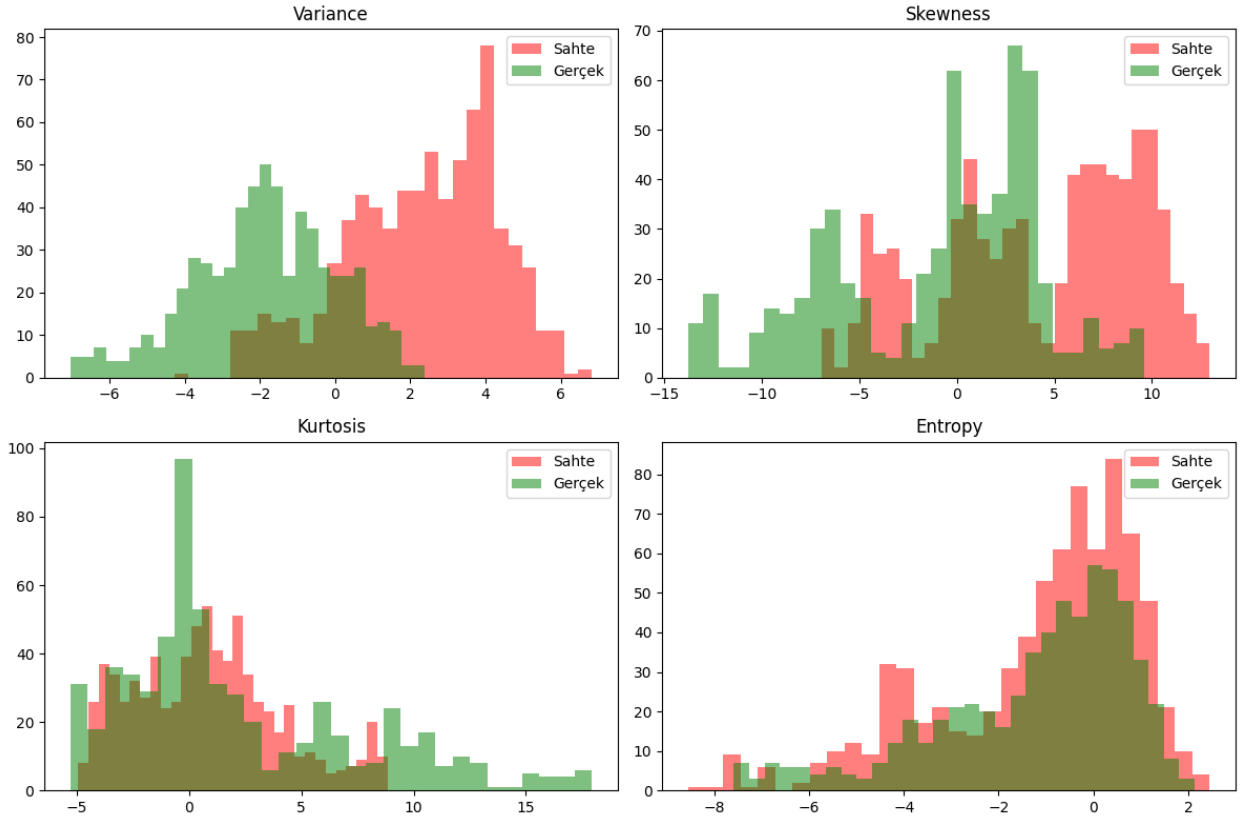
for i, col in enumerate(df.columns[:-1]): # class hariç bütun sütunlar
    plt.subplot(2, 2, i+1)  # 2 satır 2 sütun = 4 grafik

    # 0. sınıf yani sahte banknotları seçtim. histi 30'a böldüm,
değerlerini aldım
    # ve üst üste binmeler olabilir diye şeffaflık verdim
    plt.hist(df[df['Class']==0][col], alpha=0.5, label='Sahte',
bins=30, color='red')

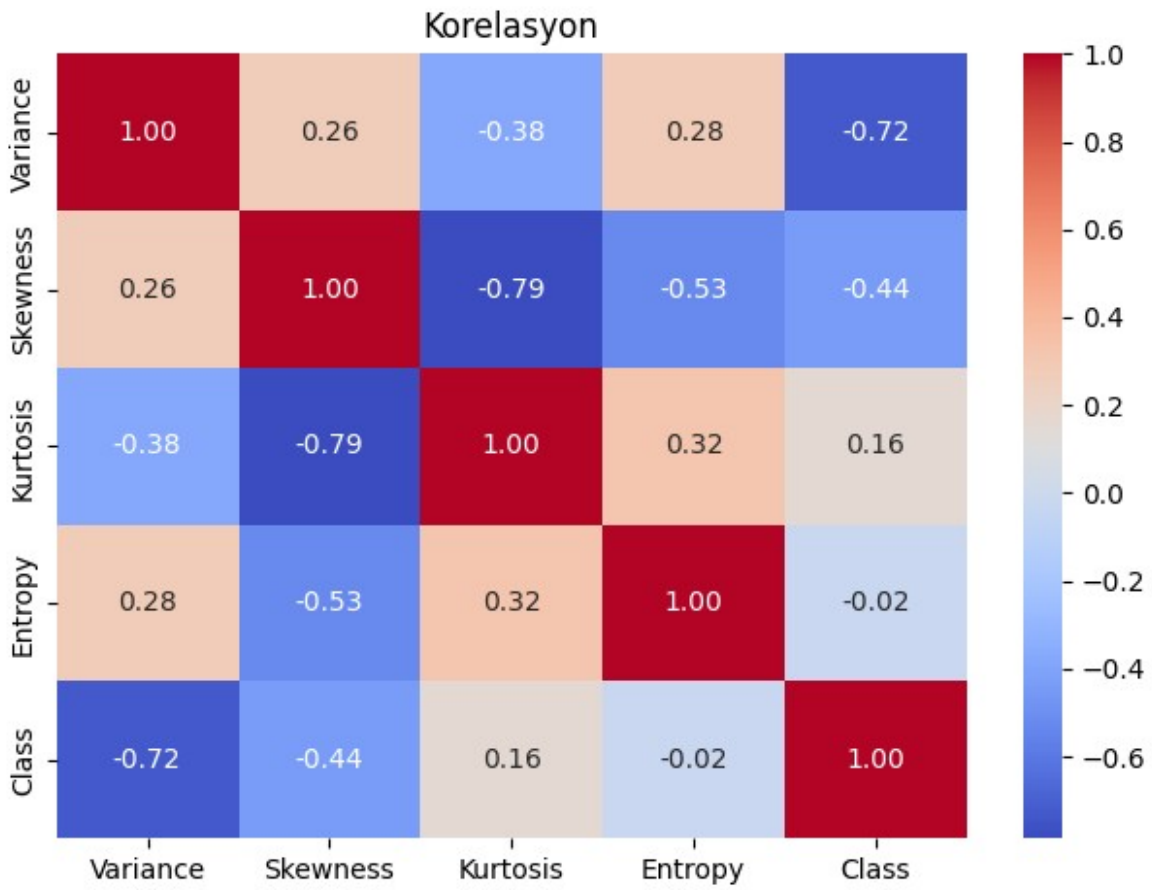
    # 1. sınıf yani gerçek banknotları seçtim. histi 30'a böldüm,
değerlerini aldım
    # ve üst üste binmeler olabilir diye şeffaflık verdim
    plt.hist(df[df['Class']==1][col], alpha=0.5, label='Gerçek',
bins=30, color='green')

    plt.title(col)
    plt.legend()

plt.tight_layout()
plt.show()
```




```
# Hangi değerlerin birbirine benzediğini veya değiştiğini görmek için
korelasyon yani ısı haritası oluşturdum.
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Korelasyon')
plt.tight_layout()
plt.show()
```




```
X = df.drop('Class', axis=1) # x yani Variance, Skewness, Kurtosis, Entropy verileri
y = df['Class'] # y yani tahmin etmek istediğimiz şey- classı tahmin etmek istiyorum. banknot sahtemi gerçekmi 0-1

# Veriyi eğitim %80 ve test %20 olarak bölüyorum
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y) #stratify=y dememin sebebi eğitim ve test setlerindeki 0 ve 1 leri aynı oranda bölsün.

print(f"Eğitim seti boyutu: {X_train.shape[0]}")
print(f"Test seti boyutu: {X_test.shape[0]}")
```

```
Eğitim seti boyutu: 1097
Test seti boyutu: 275
```

```
# Standardizasyon yapıyorum. yani veri ortalama 0, standart sapma 1
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

## Gizli Katmanlarda Neden ReLU Kullandım?

Sigmoid veya Tanh gibi fonksiyonlar değerleri çok dar bir aralığa sıkıştırdığı için, ağ derinleştikçe gradyan kaybolması problemi ortaya çıkıyor ve modelin öğrenmesi adeta felç oluyor. ReLU ise negatifleri sıfıra çevirerek çalıştığı için en azından pozitif bölgede gradyanı aynen koruyor. Bu da ağın tıkanmadan, çok daha hızlı bir şekilde öğrenmeye devam etmesini sağladı.

İşin bir de donanım ve zaman boyutu var. ReLU'nun türevini almanın matematiksel olarak son derece basit olması, modelin eğitim süresini ciddi anlamda kısalttı. Zaten kendi deney sonuçlarımla da doğruladım. Katmanlarda Sigmoid kullandığım ilk testlerde modelin doğruluk oranları beklediğimin çok altında kaldı. Aktivasyon fonksiyonunu ReLU olarak güncellediğimde ise öğrenme eğrisi bir anda stabil hale geldi ve modelin genel performansı belirgin şekilde yükseldi.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input

# Veri yapısı basit olduğu için sırayla ilerleyen Sequential modelini
kullandım.
model = Sequential([
    Input(shape=(4,)),  # Variance, Skewness, Kurtosis, Entropy 4
değer var.

    Dense(16, activation='relu'),
    # 16 nöron seçtim çünkü modelin iyi öğrenmesi istedim bu yüzden
orta değer olan 16. 4 veri 16 farklı bakış açısı.
    # activation='relu' kullandım çünkü negatifleri sıfıra çevirerek
modele doğrusal olmayan bir yapı kazandırır. öğrenmeyi kolaylaştırır.

    # overfittingi yani ezberlemeyi önlemek için rastgele %20 nöronu
kapattım.
    Dropout(0.2),

    # bilgiyi sıkıştırmak için 16 - 8e düşüyor
    Dense(8, activation='relu'),

    # tekrar dropout. ezberi önlemek için
    Dropout(0.2),

    # 0 ile 1 arası değer üretmesi için sigmoid kullandım.
    Dense(1, activation='sigmoid')
])
```

## Çıkış Katmanında Neden Sigmoid Kullandım?

Sigmoid kullanırkenki amacım modelin ürettiği sonuçları 0 ile 1 arasındaki bir değere sıkıştırmaktı. Bu sayede elde ettiğim çıktıları doğrudan birer olasılık skoru gibi yorumlayabildim. Mesela modelden 0.87 gibi bir değer aldığımda, o banknotun %87 ihtimalle gerçek olduğunu kolayca söyleyebiliyorum. Eğer burada ReLU tercih etseydim, çıktı aralığı sınırsız olacağı için

böyle bir olasılık hesabı yapmak imkansız hale gelirdi. Ayrıca problemim sadece iki sınıftan oluştuğu için Softmax gibi daha karmaşık yapılara girmedim. Sigmoid ile en sade çözüme ulaştım.

```
# model özeti
model.summary()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ dense_3 (Dense)                      │ (None, 16)                  │              80 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dropout_2 (Dropout)                  │ (None, 16)                  │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_4 (Dense)                      │ (None, 8)                   │             136 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dropout_3 (Dropout)                  │ (None, 8)                   │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_5 (Dense)                      │ (None, 1)                   │               9 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────┘
```

```
 Total params: 225 (900.00 B)

 Trainable params: 225 (900.00 B)

 Non-trainable params: 0 (0.00 B)
```

## Neden Adam Optimizer Kullandım?

Aslında deney aşamasında farklı algoritmaları da test ettim. Sondaki testler bölümünden inceleyebilirsiniz. Örneğin; SGD %50.5 gibi düşük bir doğrulukta kalırken, RMSprop %86.9 ile kağıt üzerinde en yüksek skoru verdi. Adam ise bu denemelerde %54.5 civarında kaldı.

Peki, neden en yüksek skoru veren RMSprop yerine Adam'ı seçtim? RMSprop daha yüksek skor verse de Adam çok daha dengeli bir öğrenme süreci sundu. Eğitimi gözlemledim. Adam'ın öğrenme eğrisi çok daha tutarlı bir ilerleme kaydetti. Ayrıca veri setindeki değerlerin (varyans,

entropi gibi) birbirinden farklı ölçeklerde olması, öğrenme sürecini bazen dengesizleştirebiliyor. Adam, her değer için duruma göre değişen bir öğrenme oranı belirliyor. Bu da dengesizliği önledi.

```
# model derleme
model.compile(
    optimizer=Adam(learning_rate=0.001),  # model hatasını düzeltme yöntemi, ne kadar hızlı düzelteceği
    loss='binary_crossentropy',           # tahmin gerçek değerden ne kadar uzak? tahmin doğruysa loss düşük yanlışsa loss yüksek
    metrics=['accuracy']                   # başarısı
)
```

## Neden Binary Crossentropy Loss Kullandım?

Kayıp fonksiyonu tarafında, elimideki problem bir ikili sınıflandırma olduğu için en mantıklı seçenekti. Binary Crossentropy Loss tahmin edilen olasılık ile gerçek değer arasındaki farkı ölçüyor. Model yanlış bir tahminde bulunduğunda loss değeri artıyor, doğru tahminlerde ise loss değeri azalıyor. Bu sayede model, hatayı minimize etmek için kendini güncelliyor. Aslında MSE (Ortalama Kare Hatası) gibi bir alternatifim vardı. Ama tercih etmedim. Çünkü sigmoid çıkışlı sınıflandırmalarda MSE öğrenme sürecini yavaşlatabiliyor. Ayrıca olasılık temelli yorumlamada Binary Crossentropy kadar uygun değildi.

```
# ezberlemeye başlarsa gelişmesse yani overfitting oluyorsa durdurur. Early Stopping (erken durdurma)
early_stop = EarlyStopping(
    monitor='val_loss', # validation loss (doğrulama hatası) yani ne kadar iyi genelliyor?
    patience=10, # 10 epoch boyunca iyleşmezsse dur.
    restore_best_weights=True, # son halini değil yani ezberlemeye başladığı halini değil öncesinin en iyi hali kaydedilir
    verbose=1 #konsola yazar erken durdu diye
)

# modeli eğitme
history = model.fit(
    X_train_scaled, y_train,
    epochs=100,                          # tüm veriyi en fazla 100 kez görür
    batch_size=32,                       # veriyi parçalara böler, 32 veri 1 adım. -daha hızlı ve stabil öğrenme olsun diye.
    validation_split=0.2,                # %80 öğrenme, %20 kontrol (validation)
    callbacks=[early_stop],              # model ezbere basladığında durdurmak için
    verbose=1                            # eğitilirken konsolda görmek için
)
```

```
Epoch 1/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9989 - loss:
0.0083 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 2/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9954 - loss:
0.0126 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 3/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9966 - loss:
0.0140 - val_accuracy: 1.0000 - val_loss: 0.0012
Epoch 4/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9932 - loss:
0.0153 - val_accuracy: 1.0000 - val_loss: 0.0012
Epoch 5/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9989 - loss:
0.0098 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 6/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9966 - loss:
0.0131 - val_accuracy: 1.0000 - val_loss: 0.0011
Epoch 7/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9989 - loss:
0.0106 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 8/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9943 - loss:
0.0115 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 9/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9977 - loss:
0.0073 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 10/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9954 - loss:
0.0153 - val_accuracy: 1.0000 - val_loss: 0.0011
Epoch 11/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9943 - loss:
0.0176 - val_accuracy: 1.0000 - val_loss: 0.0015
Epoch 12/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9966 - loss:
0.0106 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 13/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9977 - loss:
0.0091 - val_accuracy: 1.0000 - val_loss: 9.5363e-04
Epoch 14/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9966 - loss:
0.0089 - val_accuracy: 1.0000 - val_loss: 7.9801e-04
Epoch 15/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9989 - loss:
0.0066 - val_accuracy: 1.0000 - val_loss: 6.3157e-04
Epoch 16/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9943 - loss:
0.0108 - val_accuracy: 1.0000 - val_loss: 9.5633e-04
Epoch 17/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9989 - loss:
```

```
0.0067 - val_accuracy: 1.0000 - val_loss: 9.4196e-04
Epoch 18/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9989 - loss:
0.0110 - val_accuracy: 1.0000 - val_loss: 8.3500e-04
Epoch 19/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss:
0.0059 - val_accuracy: 1.0000 - val_loss: 6.1598e-04
Epoch 20/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9954 - loss:
0.0182 - val_accuracy: 1.0000 - val_loss: 5.9754e-04
Epoch 21/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9954 - loss:
0.0140 - val_accuracy: 1.0000 - val_loss: 6.1766e-04
Epoch 22/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9977 - loss:
0.0117 - val_accuracy: 1.0000 - val_loss: 0.0011
Epoch 23/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9989 - loss:
0.0066 - val_accuracy: 1.0000 - val_loss: 0.0012
Epoch 24/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9932 - loss:
0.0177 - val_accuracy: 1.0000 - val_loss: 8.0127e-04
Epoch 25/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9989 - loss:
0.0097 - val_accuracy: 1.0000 - val_loss: 7.6675e-04
Epoch 26/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9977 - loss:
0.0086 - val_accuracy: 1.0000 - val_loss: 7.8342e-04
Epoch 27/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss:
0.0080 - val_accuracy: 1.0000 - val_loss: 6.5736e-04
Epoch 28/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9977 - loss:
0.0103 - val_accuracy: 1.0000 - val_loss: 5.4499e-04
Epoch 29/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9954 - loss:
0.0097 - val_accuracy: 1.0000 - val_loss: 5.0171e-04
Epoch 30/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9989 - loss:
0.0071 - val_accuracy: 1.0000 - val_loss: 4.7939e-04
Epoch 31/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9977 - loss:
0.0100 - val_accuracy: 1.0000 - val_loss: 4.2727e-04
Epoch 32/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9977 - loss:
0.0061 - val_accuracy: 1.0000 - val_loss: 4.0396e-04
Epoch 33/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9989 - loss:
0.0084 - val_accuracy: 1.0000 - val_loss: 3.9260e-04
Epoch 34/100
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9954 - loss:
0.0130 - val_accuracy: 1.0000 - val_loss: 4.2305e-04
Epoch 35/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9954 - loss:
0.0161 - val_accuracy: 1.0000 - val_loss: 4.8229e-04
Epoch 36/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9977 - loss:
0.0073 - val_accuracy: 1.0000 - val_loss: 4.7273e-04
Epoch 37/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9977 - loss:
0.0064 - val_accuracy: 1.0000 - val_loss: 5.3912e-04
Epoch 38/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 1.0000 - loss:
0.0056 - val_accuracy: 1.0000 - val_loss: 5.1271e-04
Epoch 39/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 1.0000 - loss:
0.0045 - val_accuracy: 1.0000 - val_loss: 4.8914e-04
Epoch 40/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9989 - loss:
0.0058 - val_accuracy: 1.0000 - val_loss: 5.5941e-04
Epoch 41/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9966 - loss:
0.0088 - val_accuracy: 1.0000 - val_loss: 4.6677e-04
Epoch 42/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9977 - loss:
0.0054 - val_accuracy: 1.0000 - val_loss: 4.1850e-04
Epoch 43/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9966 - loss:
0.0069 - val_accuracy: 1.0000 - val_loss: 4.0361e-04
Epoch 43: early stopping
Restoring model weights from the end of the best epoch: 33.
```

## Epoch Testleri - Kaç Epoch Çalıştı? EarlyStopping kaçıncı epoch'ta durdu

```
len(history.history['loss']) # kaç epoch çalıştı

43
```

## Neden Early Stopping Kullandım?

EarlyStopping yöntemini model ezbere başladığında overfitting yaptığında yani doğrulama kaybı iyileşmediğinde eğitimi durdurmak için kullandım.

patience=10 olarak ayarladım. Ezbere başaldığında o kısımı almak yerine öncesindeki en iyi model ağırlıkları geri yüklendi.

Model genellikle: 35–45 epoch aralığında durdu.

Bu durum, modelin optimum noktaya ulaştığını ve daha fazla eğitimin overfitting riskini artıracağını gösterdi.

```
early_stop.stopped_epoch # EarlyStopping kaçıncı epoch'ta durdu #Model genellikle 35–45 epoch civarında durdu
```

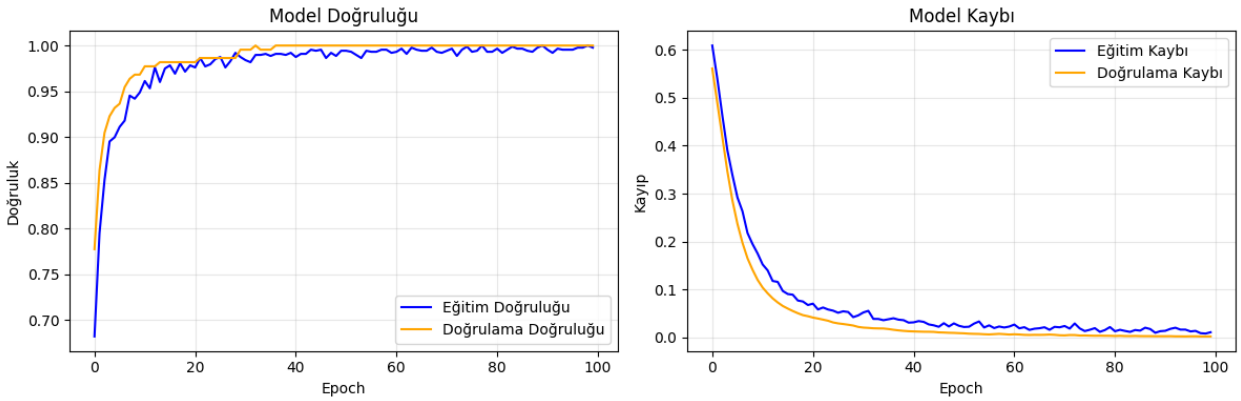
```
42
```

```
plt.figure(figsize=(12, 4)) #12x4 grafik alanı

# Doğruluk (Accuracy) Grafiği
plt.subplot(1, 2, 1) # 1 satırlık 2 sütunluk alanın 1. grafiği
plt.plot(history.history['accuracy'], label='Eğitim Doğruluğu',
color='blue')
plt.plot(history.history['val_accuracy'], label='Doğrulama Doğruluğu',
color='orange')
plt.title('Model Doğruluğu')
plt.xlabel('Epoch')
plt.ylabel('Doğruluk')
plt.legend()
plt.grid(True, alpha=0.3)

# Kayıp (Loss) Grafiği
plt.subplot(1, 2, 2) # 1 satırlık 2 sütunluk alanın 2. grafiği
plt.plot(history.history['loss'], label='Eğitim Kaybı', color='blue')
plt.plot(history.history['val_loss'], label='Doğrulama Kaybı',
color='orange')
plt.title('Model Kaybı')
plt.xlabel('Epoch')
plt.ylabel('Kayıp')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```




Eğitim sürecinde modelin doğruluk ve kayıp değerleri grafikle gösterilerek öğrenme performansı analiz edilmiştir.

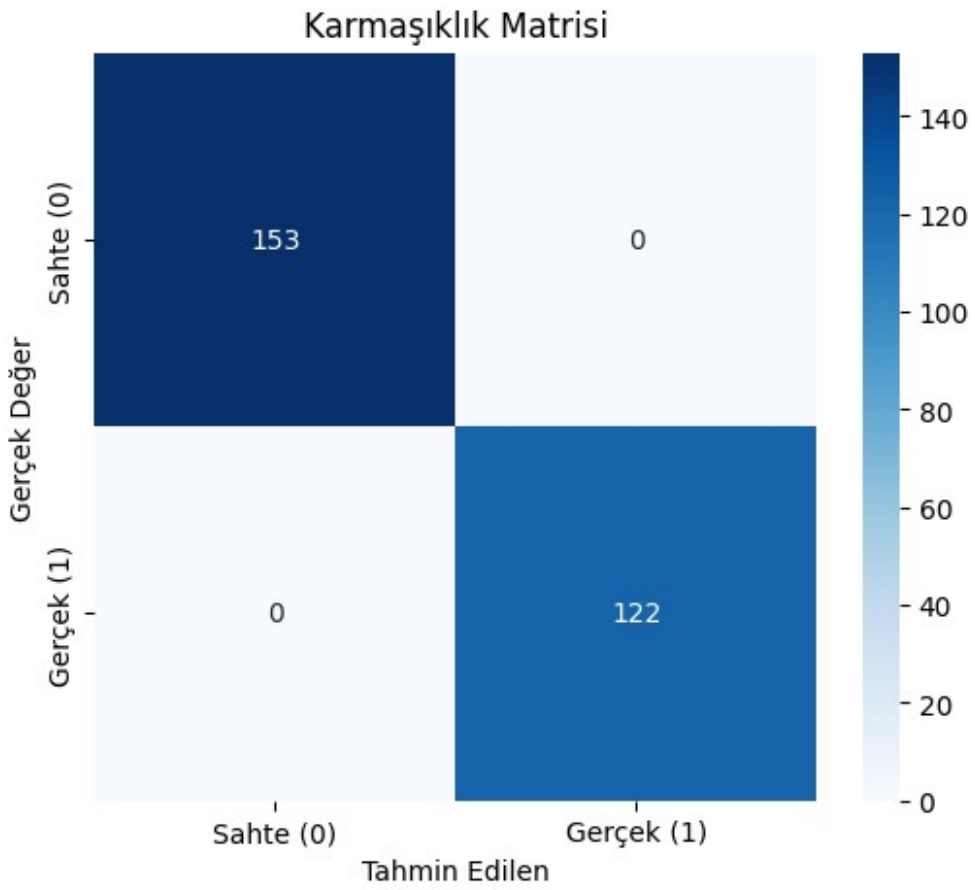
```python
# Model gerçekte ne kadar iyi çalışıyor?
test_loss, test_accuracy = model.evaluate(X_test_scaled, y_test,
verbose=0) #modeli hiç görmediği test verisiyle deniyorum
#sonuçları yazdırdım.
print(f"\n{'='*50}")
print(f"Test Kaybı (Loss): {test_loss:.4f}")
print(f"Test Doğruluğu (Accuracy): {test_accuracy:.4f}")
print(f"{'='*50}\n")

# Tahminlerin yapılması
y_pred_prob = model.predict(X_test_scaled) #model her veri için
olasılık veriyor
y_pred = (y_pred_prob > 0.5).astype(int).flatten() #olasılığı sınıfa
çeviriyorum. 0.5ten küçük eşitse sahte(0) büyükse doğru gerçek
banknot(1) oluyor.

# Karmaşıklık Matrisi (Confusion Matrix) - Gerçek & tahmin
karşılaştırması TN,TP,FN,FP
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Sahte (0)', 'Gerçek (1)'],
            yticklabels=['Sahte (0)', 'Gerçek (1)'])
plt.title('Karmaşıklık Matrisi')
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek Değer')
plt.show()
```

```
==================================================
Test Kaybı (Loss): 0.0021
Test Doğruluğu (Accuracy): 1.0000
==================================================

9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
```




```
# modelin başarısını diğer metriklerle de Precision, Recall, F1-score, Support gösterme
print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_pred, target_names=['Sahte', 'Gerçek']))


Sınıflandırma Raporu:
              precision    recall  f1-score   support

       Sahte       1.00      1.00      1.00       153
      Gerçek       1.00      1.00      1.00       122

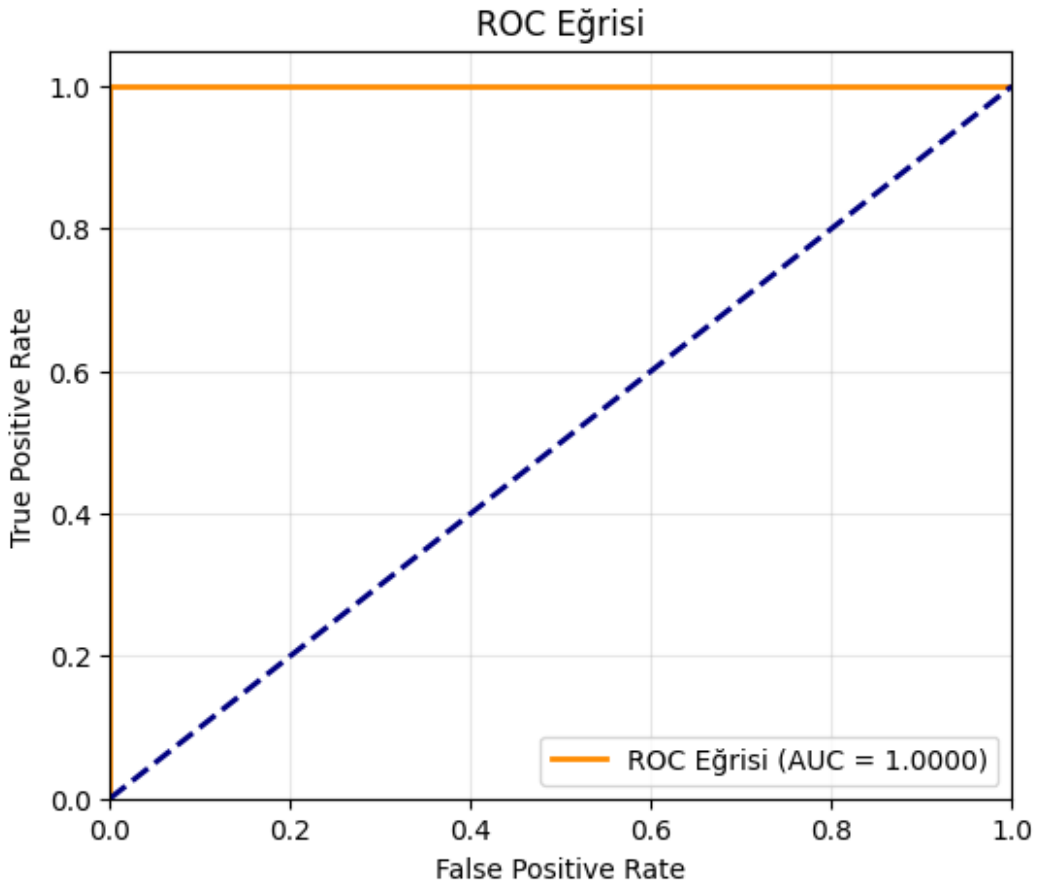
    accuracy                           1.00       275
   macro avg       1.00      1.00      1.00       275
weighted avg       1.00      1.00      1.00       275


# ROC Eğrisi -modelin davranış eğrisi  ve AUC Skoru - genel başarı gücü, ROC eğrisinin altındaki alan
roc_auc = roc_auc_score(y_test, y_pred_prob) #modelin gerçeği ve sahteyi ne kadar iyi ayırdığını ölçer
fpr, tpr, _ = roc_curve(y_test, y_pred_prob) #True Positive Rate -
```

```
False Positive Rate | Gerçekleri doğru bulma oranı ile yanlış gerçek deme oranı

#roc eğrisi çizimi
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Eğrisi (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Eğrisi')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.show()

print(f"ROC-AUC Skoru: {roc_auc:.4f}")
```




```
ROC-AUC Skoru: 1.0000
```

# Testler

## Optimizer Testleri

```
def create_model():
    model = Sequential([
        Dense(16, activation='relu',
input_shape=(X_train_scaled.shape[1],)),
        Dense(8, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
    return model

model_adam = create_model()

model_adam.compile(
    optimizer=Adam(0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_adam = model_adam.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

adam_loss, adam_acc = model_adam.evaluate(X_test_scaled, y_test,
verbose=0)
print("Adam Accuracy:", adam_acc)
```

```
c:\Users\LightPath\.continue\sahte\mlenv\Lib\site-packages\keras\src\
layers\core\dense.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 1.
Adam Accuracy: 0.5454545617103577
```

```
model_sgd = create_model()

model_sgd.compile(
    optimizer=SGD(0.01),
```

```
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_sgd = model_sgd.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

sgd_loss, sgd_acc = model_sgd.evaluate(X_test_scaled, y_test,
verbose=0)
print("SGD Accuracy:", sgd_acc)
```

```
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 1.
SGD Accuracy: 0.5054545402526855
```

```
model_rms = create_model()

model_rms.compile(
    optimizer='rmsprop',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_rms = model_rms.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

rms_loss, rms_acc = model_rms.evaluate(X_test_scaled, y_test,
verbose=0)
print("RMSprop Accuracy:", rms_acc)
```

```
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 1.
RMSprop Accuracy: 0.8690909147262573
```

```
import pandas as pd

results = pd.DataFrame({
    "Optimizer": ["Adam", "SGD", "RMSprop"],
    "Accuracy": [adam_acc, sgd_acc, rms_acc]
```

```
})

print(results)
```

```
  Optimizer  Accuracy
0      Adam  0.545455
1       SGD  0.505455
2   RMSprop  0.869091
```

## Dropout Var / Dropout Yok Testleri

Dropout katmanını modelin overfittingi ezberi yapmasını önlemek için kullandım. Dropout eklendiğimde modelin test performansı arttı. Genelleme kabiliyeti güçlendi. Eğitim ve test doğrulukları arasındaki fark azaldı.

```
#drop YOK

def create_model_no_dropout():
    model = Sequential([
        Dense(16, activation='relu',
input_shape=(X_train_scaled.shape[1],)),
        Dense(8, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
    return model

model_a = create_model_no_dropout()

model_a.compile(
    optimizer=Adam(0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_a = model_a.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

train_loss_a, train_acc_a = model_a.evaluate(X_train_scaled, y_train,
verbose=0)
test_loss_a, test_acc_a = model_a.evaluate(X_test_scaled, y_test,
verbose=0)
```

```
print("Dropout Yok - Train Acc:", train_acc_a)
print("Dropout Yok - Test Acc:", test_acc_a)
```

```
c:\Users\LightPath\.continue\sahte\mlenv\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 1.
Dropout Yok - Train Acc: 0.7456700205802917
Dropout Yok - Test Acc: 0.7599999904632568
```

```
#DROPOUT VAR
from tensorflow.keras.layers import Dropout

def create_model_with_dropout():
    model = Sequential([
        Dense(16, activation='relu', input_shape=(X_train_scaled.shape[1],)),
        Dropout(0.2),

        Dense(8, activation='relu'),
        Dropout(0.2),

        Dense(1, activation='sigmoid')
    ])
    return model

model_b = create_model_with_dropout()

model_b.compile(
    optimizer=Adam(0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_b = model_b.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

train_loss_b, train_acc_b = model_b.evaluate(X_train_scaled, y_train, verbose=0)
```

```
test_loss_b, test_acc_b = model_b.evaluate(X_test_scaled, y_test,
verbose=0)

print("Dropout Var - Train Acc:", train_acc_b)
print("Dropout Var - Test Acc:", test_acc_b)
```

```
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 1.
Dropout Var - Train Acc: 0.8322698473930359
Dropout Var - Test Acc: 0.8836363554000854
```

```
import pandas as pd

results = pd.DataFrame({
    "Model": ["Dropout Yok", "Dropout Var"],
    "Train Accuracy": [train_acc_a, train_acc_b],
    "Test Accuracy": [test_acc_a, test_acc_b]
})

print(results)
```

```
         Model  Train Accuracy  Test Accuracy
0  Dropout Yok         0.74567       0.760000
1  Dropout Var         0.83227       0.883636
```

# Sonuçlar ve Performans Analizi

## Optimizer Karşılaştırması

| Optimizer | Accuracy |
|---|---|
| Adam | 0.5455 |
| SGD | 0.5055 |
| RMSprop | 0.8691 |

En iyi performansı **RMSprop optimizer** göstermiştir. Adam ve SGD daha düşük doğrulukta kalmıştır. Bu durum veri setine göre kendini ayarlayabilen optimizasyon yöntemleri daha iyi çalıştı demektir.

---

## Dropout Etkili Oldu mu?

| Model | Train Accuracy | Test Accuracy |
|---|---|---|
| Dropout Yok | 0.7457 | 0.7600 |
| Dropout Var | 0.8323 | 0.8836 |